



PDF Download
3719027.3765195.pdf
26 February 2026
Total Citations: 0
Total Downloads: 1694

Latest updates: <https://dl.acm.org/doi/10.1145/3719027.3765195>

RESEARCH-ARTICLE

Passwords and FIDO2 Are Meant To Be Secret: A Practical Secure Authentication Channel for Web Browsers

ANUJ GAUTAM, The University of Tennessee, Knoxville, Knoxville, TN, United States

TARUN YADAV, Brigham Young University, Provo, UT, United States

GARRETT SMITH, Brigham Young University, Provo, UT, United States

KENT SEAMONS, Brigham Young University, Provo, UT, United States

SCOTT RUOTI, The University of Tennessee, Knoxville, Knoxville, TN, United States

Open Access Support provided by:

The University of Tennessee, Knoxville

Brigham Young University

Published: 22 November 2025

[Citation in BibTeX format](#)

CCS '25: ACM SIGSAC Conference on
Computer and Communications Security
October 13 - 17, 2025
Taipei, Taiwan

Conference Sponsors:
SIGSAC

Passwords and FIDO2 Are Meant To Be Secret: A Practical Secure Authentication Channel for Web Browsers

Anuj Gautam
University of Tennessee
Knoxville, TN, USA
agautam1@alum.utk.edu

Tarun Yadav
Brigham Young University
Provo, UT, USA
tarun14110@iitd.ac.in

Garrett Smith
Brigham Young University
Provo, UT, USA
gds33@byu.edu

Kent Seamons
Brigham Young University
Provo, UT, USA
seamons@cs.byu.edu

Scott Ruoti
University of Tennessee
Knoxville, TN, USA
ruoti@utk.edu

Abstract

Password managers provide significant security benefits to users. However, malicious client-side scripts and browser extensions can steal passwords after the manager has autofilled them into the web page. In this paper, we extend prior work by Stock and Johns, showing how password autofill can be hardened to prevent these local attacks. We implement our design in the Firefox browser and conduct experiments demonstrating that our defense successfully protects passwords from XSS attacks and malicious extensions. We also show that our implementation is compatible with 97% of the Alexa top 1000 websites. Next, we generalize our design, creating a second defense that prevents recently discovered local attacks against the FIDO2 protocols. We implement this second defense into Firefox, demonstrating that it protects the FIDO2 protocol against XSS attacks and malicious extensions. This defense is compatible with all websites, though it does require a small change (2–3 lines) to web servers implementing FIDO2.

CCS Concepts

• **Security and privacy** → **Authentication; Browser security; Web application security.**

Keywords

Password entry, password managers, browser security, authentication

ACM Reference Format:

Anuj Gautam, Tarun Yadav, Garrett Smith, Kent Seamons, and Scott Ruoti. 2025. Passwords and FIDO2 Are Meant To Be Secret: A Practical Secure Authentication Channel for Web Browsers. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3765195>

1 Introduction

Password managers seek to improve the security of passwords by helping users generate, store, and enter strong passwords [36, 48].



This work is licensed under a Creative Commons Attribution 4.0 International License. *CCS '25, Taipei, Taiwan*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3765195>

However, password manager security has a conspicuous gap—passwords are vulnerable to theft after they are autofilled into the browser and before they are transmitted to the website. This includes theft from web trackers [46], injection attacks (e.g., XSS) [45], malicious browser extensions [25, 42, 51], or compromised JavaScript libraries [18, 38] (i.e., a supply chain attack). With the relative widespread nature of these issues [14, 41, 50], this security gap urgently needs to be closed.

A decade ago, Stock and Johns [49] proposed modifying password managers so that instead of autofilling the user's password, the extension would fill a random value (i.e., a nonce). Then, after the page is submitted but before the related HTTP request is transmitted over the network, the nonce is replaced with the actual password. Critically, the actual password is never visible to the webpage, preventing DOM-based attacks. Unfortunately, even at the time this protocol was proposed, it was incompatible with modern browser design [7], and the one browser where it did work, Firefox, removed this functionality shortly after [33].

Addressing this limitation, this paper's first contribution is practical—we describe how modern browsers can be modified to add a nonce-based password replacement API. Our API limits the extent to which browser requests can be modified and provides safeguards that allow password managers to ensure passwords are sent only to the appropriate destination. To demonstrate the feasibility of this design, we forked and modified the Firefox browser to implement our API and Bitwarden to use it. We empirically evaluated the security of our implementations, demonstrating that they stop password exfiltration by DOM-based adversaries (e.g., web trackers, XSS, compromised JavaScript libraries). We also evaluated our API on 573 sites with login forms from the Alexa Top 1000, showing that it is compatible with 97% of those sites.

Our work has substantial practical and research implications. First, it represents an immediate and significant step forward for the security of password-based authentication for tens of millions of password manager users. Second, it allows the research community to build on and expand both our API and the protocol proposed by Stock and Johns. In the decade since Stock and Johns' original paper, there has been no research expanding on this promising work [36], likely because it was non-functional. By providing a functional implementation of Stock and Johns' protocol on modern browsers, researchers can explore how to extend or use our API to further

secure passwords (see §8.3) or other browser-server operations (see §8.2).

Our second contribution is that we identify a gap in Stock and Johns's original threat model: malicious browser extensions that can exfiltrate the password by examining outgoing HTTP requests. In an analysis of extensions on the Chrome web store, we identified thousands of extensions with sufficient permissions to conduct such an attack. We then demonstrate how our defense can be modified to prevent this new class of attacks.

Inspired by our API, we hypothesized that similar APIs could be used to harden other browser-server services. Our paper's third contribution is that we prove this hypothesis correct by developing a second defense that secures FIDO2 against recently discovered local attacks—i.e., DOM-based attacks (e.g., XSS, compromised JavaScript libraries) and malicious extensions [19, 22, 32, 56]. This second API provides a confidential channel for the web server and the client's FIDO2 device to communicate without allowing the web page's DOM or browser extensions to access sensitive values. This design requires no changes for FIDO2 devices and only a trivial change (2–3 lines) to the web servers implementing the FIDO2 protocol. We implement this API into Firefox and evaluate it empirically.

2 Background

In this section, we provide background material on the password autofill workflow, how browsers process forms and web requests, a DOM-centric threat model for password exfiltration, and Stock and Johns' proposal for addressing DOM-based password exfiltration [49].

2.1 Password Autofill Workflow

The password autofill workflow includes the following steps:

- (1) The user visits a webpage with a form containing an input element to enter their password.
- (2) The password manager detects this form and offers to autofill the password. It does so by providing the user with a list of passwords associated with the current domain that they can autofill.
- (3) The user selects which password to autofill.¹
- (4) The password manager sets the value field of the password input element. At this point, the password is stored within the web page's DOM.
- (5) The user submits the form.
- (6) The browser creates a `webRequest` object responsible for sending the password to the server.
- (7) The password is transferred over the network by the browser (preferably using a TLS connection).

This flow also applies to account creation, with the same general process for entering and transmitting passwords.

2.2 Browser Functionality

2.2.1 Browser Form Submission. The form submission process is defined in the HTML specifications [53] and starts when the

¹This process is automated in some password managers, though it is unsafe to do so [36].

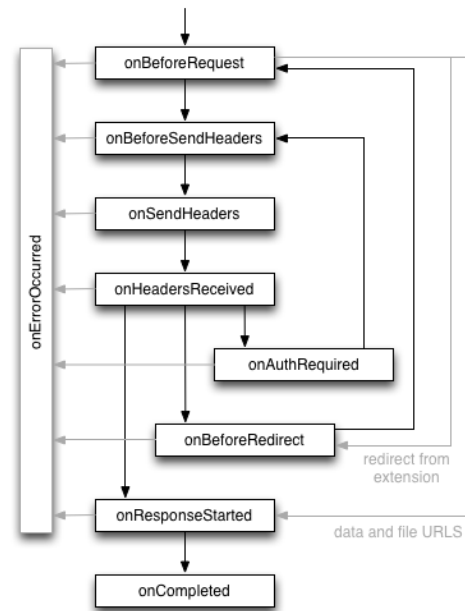


Figure 1: Web request API flow [6]

`submit()` method is called on a `FormElement` object. This can happen when the user clicks a submit button associated with that form or when JavaScript directly calls this method.

First, the browser will construct an entry list consisting of all the input fields in the form [52]. This process involves sanitizing textual inputs and field names and converting checkboxes and radio selections into strings representing the selected items. Next, the form data is encoded for network transmission based on the form's `accept-charset` attribute. The browser will then convert all values into name-value pairs, applying URL encoding if required by the `accept-charset` attribute. These items are all put into `FormSubmission` object. Finally, the browser will create a `webRequest` object that sends a web request to the server. The web request's destination is set using the form's `action` property. The body of the web request is a collection of name-value pairs encoded by the `FormSubmission` object.

2.2.2 Browser `webRequest` API. The `webRequest` API is a browser extension-only API that lets extensions read, modify, or cancel web requests and responses. To access this API, extensions register event listeners for one or more stages of the web request processing lifecycle (see Figure 1).

In this lifecycle, the `onBeforeRequest`, `onBeforeSendHeaders`, and `onSendHeaders` events happen before the request is sent and provide access to the request body, allowing passwords included there to be exfiltrated. The `onBeforeRequest` and `onBeforeSendHeaders` allow extensions to redirect or cancel the request, while the latter also allows modification of request headers. The remaining stages focus on what happens after the request has been submitted. Since they lack access to the request body, they are less important in the context of secure password entry.

Critically, none of these stages allow the request body to be modified. By the time `onBeforeRequest` is reached, the form submission process has already created the `FormSubmission` object, and the extensions are only provided with a read-only copy of the data in this object (not the object itself). The reason behind not allowing the request body to be changed is unknown, but requests to add this functionality have gone unfilled over the last decade, and there is little evidence that it will be added [7, 33]. This keeps the protocol that Stock and Johns described from working with modern browsers [49].

2.3 DOM-Centric Threat Model

Stock and Johns' did not provide a threat model for their defense. As such, we have created the following threat model for DOM-based password exfiltration.

2.3.1 Attack Vector. After a password is entered or autofilled into a webpage, that password is accessible as part of the DOM to JavaScript running on the page. This JavaScript can be sourced from the webpage itself, JavaScript libraries loaded by the webpage, JavaScript injected by browser extensions, or JavaScript injected using an attack (e.g., cross-site scripting).

2.3.2 Adversaries. Based on the above two attack vectors, we have identified three local adversaries:

DOM Observer. A DOM observer is an entity that the webpage itself has given access to the DOM to enable some desired functionality, such as serving ads or providing a JavaScript library. As such, this adversary has full control of the webpage's DOM (attack vector 1) and can read and modify it. While this entity does not intend to exfiltrate passwords, during normal operations, it may scan the DOM, unintentionally access the password, and send it back to a non-authentication-related endpoint [46]. While this adversary may not be interested in the passwords themselves, other attackers can attempt to steal them from this entity if they store them (e.g., in their logs). This entity is not intentionally malicious, so it will not attempt to circumvent password exfiltration defenses.

A good example of this type of adversary is web trackers. Web trackers are scripts that track users across different websites to serve advertisements more effectively. Trackers collect various information and user interactions without the user knowing about them. Senol et al. [46] analyzed form submissions on 100,000 popular websites, investigating whether web trackers on those pages would exfiltrate user passwords. They find that at least 2–3% of those pages include web trackers that exfiltrate passwords. While the motivations for this exfiltration are largely unknown, in many cases, it likely represents an honest-but-curious attacker model—i.e., web trackers are simply grabbing whatever information they can and are not targeting passwords specifically. Regardless, as shown by Dambra et al. [15], users encounter these web trackers frequently, indicating a need to protect user passwords against these honest-but-curious trackers.

Malicious DOM Exfiltrator. This adversary has full control of the webpage's DOM (attack vector 1) and can read and modify it. They have gained this access by inserting a malicious client-side script

into the webpage. They aim to leverage this access to retrieve the user's password from within the DOM. They will succeed at their goal if the password is ever included in the webpage's DOM. However, this adversary's capabilities are limited by security primitives built into the browser—they cannot violate the same origin policy (e.g., directly access passwords stored in the password manager) and do not have access to view the web requests created by the browser.

The most common source of malicious client-side scripts is cross-site scripting (XSS) attacks. In these attacks, an adversary coerces a website to include attacker-controlled scripts in a web page's DOM, either by tricking users into clicking a link with the malicious script (reflected XSS attack) or uploading the malicious script to the website (stored XSS attack). While defenses for XSS attacks are well known, the OWASP foundation consistently ranks them in its top 10 web application security risks [41], and hundreds of XSS attacks have been reported in January of 2024 alone [14].

Another common source of malicious client-side scripts are websites that use third-party libraries, which are ubiquitous [28]. While libraries produced by an adversary are transparently dangerous, more concerning are supply chain attacks [18, 38]. In these attacks, an adversary compromises an otherwise benevolent software library, which then compromises websites that rely on this library when it is updated. These attacks are already somewhat common [18, 38], with WhiteSource [54] identifying 1300 malicious JavaScript libraries in 2021. Also, in 2024, an attack against a popular JavaScript library injected malicious code into over 100 thousand websites [50].

While supply chain attacks are problems for client-side and server-side libraries, this paper is primarily concerned with client-side supply chain attacks. Web client-side supply chain attacks are especially concerning as it is common to load libraries from external sources.² If a website adopts this practice, it will be compromised immediately as soon as the library is compromised, without needing to wait for the website to explicitly update to the compromised version of the library.

2.4 Nonce-Based Password Replacement

To address DOM-based password exfiltration, Stock and Johns [49] proposed a nonce-based password replacement protocol. In this protocol, password manager browser extensions autofill a random password (i.e., a nonce) instead of the actual password. It is this random password that DOM-based scripts have access to, not the actual password. The password manager also monitors all outgoing POST requests, looking for password nonces and replacing them with the actual password.

3 A Secure Browsers Channel for Password Autofill

Unfortunately, even at the time Stock and Johns' proposed their nonce-based password replacement protocol, it was incompatible with modern browser design [7], and the one browser where it did work, Firefox, removed this functionality shortly after [33]. Moreover, there was no obvious path for getting this protocol to work in browsers, short of the daunting task of figuring out how

²For example, see usage instructions for Bootstrap at <https://getbootstrap.com/>.

to modify the browsers themselves. This reality prevented this protocol from being able to be adopted by password managers [36]. Additionally, it kept researchers from being able to build on this idea.

Our paper’s first contribution is addressing these challenges by describing how browsers can be modified to provide a nonce-based password replacement API. Implementing this API was non-trivial, requiring hundreds of hours of research and development work. Not only does our contribution provide a direct benefit to password manager users, but it also benefits the research community by providing a working implementation that they can build on to continue exploring what is possible with nonce-based replacement defense—something that was not previously possible, but for which there are many areas of compelling future work (see §8.3).

3.1 API Design

Our API is an extension of the Web Request API found in all modern browsers, adding a new stage to the `webRequest` API that supports nonce-based password replacement: `onRequestCredentials`. This stage occurs before `onBeforeRequest`. This new stage provides a secure browser channel for transmitting passwords between password managers and websites without allowing any DOM-based scripts access to the content.

To use this channel, a password manager will first autofill a random password nonce into the webpage. At the same time, it will register a callback handler with `onRequestCredentials`. This handler will be called during page submissions, receiving read-only details about the web request. Using these details, the password manager will determine if replacing the password nonce with the actual password is safe. Based on recommendations from the research literature [30, 35, 36, 47, 49] and to prevent a reflection attack discussed in §5.1.1, our design requires password managers to make the following safety checks:

- (1) Check that the web page is not displayed in an `iFrame`.
- (2) Check that the submission channel does not use HTTP or an insecure HTTPS connection.
- (3) Check that the origin (protocol, domain, port) matches the origin identified by the password manager entry being autofilled. Even better, the password manager can store the exact URL where credentials should be submitted, only submitting passwords to that URL.³
- (4) Check that the nonce is not in the GET parameters.
- (5) Check that the name of the field storing the nonce matches the name of the field where the password nonce was autofilled. Additionally, this field should have a valid name, such as `password` [35, 36].

If the handler determines a substitution should take place, it returns a tuple with the following data: (i) the name of the field that was autofilled with the password nonce, (ii) the password nonce, (iii) the replacement password, (iv) the origin (scheme, host, port) associated with the password. After receiving this data from

registered handlers, the browser will then search through the key-value pairs in the `FormSubmission` object (see §2.2.1), making the requested substitution if and only if (a) the indicated field’s value matches the password nonce exactly, (b) the web request will be sent to the indicated origin. While the password manager should already have checked these details, we have the browser also check them to provide some level of defense in depth. Because replacement happens within the `FormSubmission` object, the browser handles proper sanitization and encoding of the key-value pairs. From this point on, the browser processes the web request as normal.

The above flow is visualized in Figure 2. Looking at this flow, it is clear that the password nonce is ever available within the DOM, preventing password theft by either the *DOM Observer* or the *Malicious DOM Exfiltrator*.

3.2 API Implementation

To validate our design’s feasibility, security, and deployability, we implemented it into Mozilla Firefox 107.0 and the Bitwarden password manager. Below, we describe the implementation details for injecting and replacing password nonces.

3.2.1 Step 1: Injecting the Nonce. First, the password manager will register two callbacks, one for `onBeforeRequest` and one for the new `onRequestCredential`. Next, as needed, the password manager will autofill a nonce in place of a password. After doing so, it will internally store an association between the web page and (i) the nonce, and (ii) the name of the field storing the nonce, and (iii) the password manager entry that is being autofilled

3.2.2 Step 2: Validating the Replacement. Eventually, the page will be submitted, and a `webRequest` will be created by the browser, with this `webRequest` containing the autofilled nonce. This will cause the callback registered with `onBeforeRequest` to be triggered. Within this callback, the password manager can see the details of the `webRequest`, including the destination URL, the HTTP method used, and the request body. Using this information, the password manager determines if (a) there is a nonce associated with the web page that generated the `webRequest`, (b) the nonce is in the request body, and (c) replacing the nonce would be safe. If all these checks pass, the password manager will internally store an association between the `webRequest` and (i) the nonce, and (ii) the name of the field storing the nonce, and (iii) the password manager entry that is being autofilled

3.2.3 Step 3: Executing the Replacement. Immediately before sending the `webRequest` over the wire, the callback registered with `onRequestCredential` will be triggered. This callback once again receives request details as input, with the request body stripped out. In this step, if the password manager has a nonce associated with the current `webRequest`, it will simply return the associated nonce, password, field storing the nonce, and the URL to which the password manager expects the password to be submitted.

The password manager will then use this information to locate and replace the nonce in the request body. To prevent unintended (or malicious) consequences from replacing the nonce, the browser will only make this substitution if,

³Determining the exact URL could be done by having password managers store associations for popular websites, storing prior successful submission locations, or crowdsourcing the creation of associations.

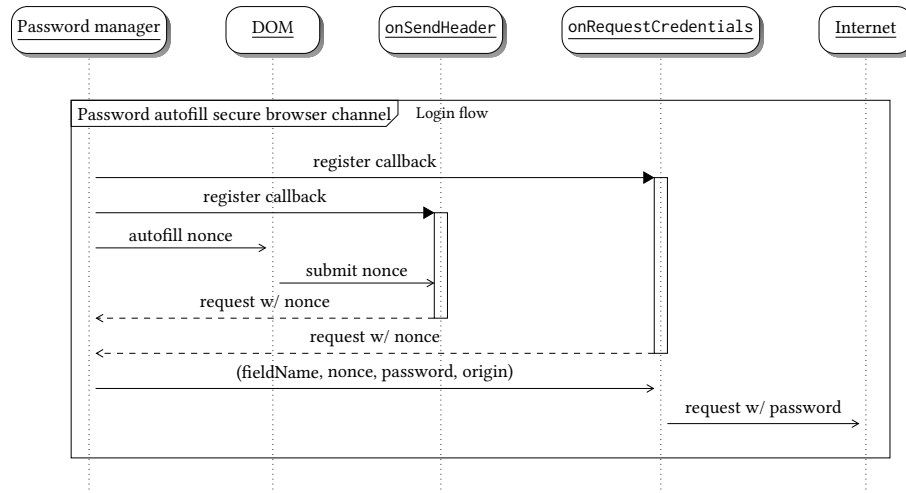


Figure 2: A flow diagram for our password autofill secure browser channel

- i) The field name storing the nonce exactly matches the field name specified by the password manager.
- ii) The field value exactly matches the nonce with no additional characters.
- iii) The submission URL matches the origin (protocol, domain, port) of the URL provided by the password manager.

While the password manager should have already checked these items before submitting the password to be replaced, we have the browser also check these items to ensure they will be checked. This is an important form of defense in depth [35]. If any of these checks fail, no substitution is made, and the request will be sent still containing the nonce.

4 Handling Malicious Extensions

As we designed our API, we identified a limitation in the threat model used by Stock and Johns’ protocol—it did not consider malicious browser extensions. While such extensions were rare when Stock and John’s paper was published, they have become a significant problem in our modern day [25, 42, 51]. As such, our paper’s second contribution is providing an extension to the base threat model to account for malicious extensions. We also describe how our API was designed to protect against this class of attacker.

4.1 Malicious Extension Threat Model

4.1.1 Attack Vectors. In addition to exfiltrating the password from the DOM, a malicious extension can also try to exfiltrate the password from an outgoing web request. When the form containing the password is submitted, the browser creates a web request to transmit the password to the server. Browser extensions with permissions for the `webRequest` API can read the content of outgoing web requests, exfiltrating it there.

4.1.2 Adversary—Web Request Exfiltrator. This adversary can view web requests and their associated data before the browser transmits them. They have gained this access by tricking the user into installing a malicious browser extension or compromising an

extension the user has already installed. Their goal is to exfiltrate passwords from the web request’s body just before transmission. This attacker may also be able to inject arbitrary JavaScript into the webpage. However, as this capability is already covered by the malicious DOM exfiltrator, for this attacker, we focus primarily on their ability to read web requests.

There are many instances of malicious browser extensions used by millions of users on official Chrome/Firefox extension stores. In 2017, eight extensions, used collectively by over 4.8 million users, were compromised and used to steal login credentials [34]. In 2020, 500 Chrome browser extensions were discovered secretly uploading private browsing data to attacker-controlled servers and redirecting victims to malware-laced websites [40]. Also, Awake has identified 111 malicious Chrome extensions that take screenshots, read the clipboard, harvest password tokens stored in cookies or parameters, grab user keystrokes (like passwords), etc. [2]. These extensions have been downloaded over 32 million times. Finally, in 2021, Cato’s analysis of network data showed that 87 out of 551 unique Chrome extensions used on customer networks were malicious [5]. *In addition to inherently malicious extensions, extensions can be compromised through supply chain attacks, as already discussed for the DOM attacker.*

There are limitations on what a malicious browser extension can do. Most importantly, all extensions are given a unique origin, preventing one extension from accessing the data or scripts of another extension. As such, the browser’s same-origin policy prevents malicious extensions from directly accessing each other’s data or scripts. The only way for a malicious extension to gain access to the passwords stored by a password manager is for that manager to copy those passwords to an origin where the malicious extension has access (i.e., the webpage).

The second limitation is that for a malicious extension to exfiltrate passwords, it needs to be granted one or more of the following permissions [13]:

- The scripting permission allows the injection of client-side scripts on any webpage [10].

- The `activeTab` permission allows the injection of client-side scripts on the root webpage of the actively focused tab [8].
- The `content script` attribute injects a specified client-side script into webpages with a matching origin (wildcards are allowed) [12].
- The `declarativeNetRequest` permission allows the modification of web response bodies, which can be used to inject client-side scripts [9].
- The `webRequest` permission allows reading web request bodies (which can contain passwords) [11].

For an extension to be granted these permissions, the user must approve the grant during installation or when the permission is first used. However, research has shown that users struggle to understand these permissions and are likely to grant them without much consideration [27]. Still, permissions are useful—if an extension is compromised through a supply chain attack, the attacker will not be able to change the manifest and will be constrained by the extension’s existing permissions.

To understand the prevalence of these permissions in existing extensions, we collected 101,414 browser extensions from the Chrome webstore between August 28, 2022, and September 13, 2022. We then extracted and analyzed their manifest files. We also estimate user counts based on data fetched from CRXcavator⁴ (pulled May 03, 2023).

We identified 12,576 extensions that can inject client-side scripts on any webpage, including 55 with at least 500,000 users. We also identified 4,169 extensions that can read any web request’s body, including 19 with at least 500,000 users. Of these extensions, 1,410 can read any web request’s body but not inject client-side scripts, including 6 with at least 500,000 users. While it is unclear how many of these extensions are malicious (the percentage is likely small), it is not unheard of, even for extensions with many downloads [51]. Moreover, these extensions are vulnerable to compromise through supply-chain attacks, which are becoming increasingly common.

4.2 API Design

In the original design of our API, malicious extensions can access the password as it will be visible in the `requestBody` after it is replaced in `onRequestCredentials`. To address this flaw, we make two changes to our API. First, we move `onRequestCredentials` after all stages where `requestBody` is accessible: after `onSendHeaders` and before `onHeadersReceived`. Second, when presenting extensions with the `requestBody` in `onRequestCredentials`, we show `requestBody` as it was before any substitutions were processed. This prevents malicious extensions from learning about the password manager’s replacement value (i.e., the password) during `onRequestCredentials`. Moreover, since this is the last stage where `requestBody` is accessible using the `webRequest` API, they will not have an opportunity to view the modified `requestBody`.

4.3 API Implementation

In our implementation, we slightly deviated from the design described above. Instead of having the web request’s body available in the `onRequestCredentials` callback, we stripped this

data entirely from callbacks at this stage. This necessitated moving the password manager’s code, validating whether a replacement should occur into the `onBeforeRequest` event, which has access to the request body. Replacement still occurs within the `onRequestCredentials` stage, meaning that our implementation still protects against password exfiltration from browser extensions.

This deviation arose from an implementational challenge around creating copies of the request body, which is necessary to prevent malicious extensions from seeing replaced passwords during the `onRequestCredentials` stage. As this only further restricts information available to extensions, it does not negatively impact security. We eventually resolved the challenge that prevented us from implementing our solution exactly as described in our design; however, by then, we had already conducted a thorough evaluation of its functionality and performance, so we described the implementation as it existed when conducting this testing.

5 Evaluation

5.1 Security

By design, our API prevents DOM-based and `webRequest`-based exfiltration. The password manager never injects the nonce into the DOM, and JavaScript running within a web page has no access to the `webRequest` API. As such, there is no way for DOM-based adversaries to access the secret. Similarly, as `onRequestCredential` is the final stage where the `requestBody` is available, and it does not share the modified `requestBody` with extensions, malicious extensions are unable to access the substituted password.

Still, to provide even more confidence in our defense, we also tried to conduct DOM-based attacks to see if there were any mistakes in our API or password manager’s implementation. To this end, we created a basic PHP login page that takes in a username and password as form input and logs the credentials in plaintext for verification purposes. To test this web page, we would autofill with the password and then submit the web page. To simulate a DOM-based attack, we would inject arbitrary JavaScript on the test web page, attempting to scrape the password field before page submission. To simulate extension-based attacks, we built extensions with full access to the `webRequest` API, including the new `onReplaceCredential` callback.

As expected, these efforts yielded no fruit. Still, they were valuable as they helped us identify a potential attack that can occur if the password manager does not check the destination URL where the password will be sent—a *reflection attack*. This attack is prevented by following the safety checks described in §3.1.

5.1.1 Reflection Attack. An attacker aware that password nonces are being used could attempt a reflection attack. In this attack, they would do the following:

- (1) Change the password form to submit to a page that displays (i.e., reflects) one or more of the submitted form fields. This page needs to be within the same origin to trigger replacement.

⁴<https://crxcavator.io/>

- (2) Change the name of the password field to one of the fields that will be reflected.
- (3) Cause the form to be submitted.
- (4) Read the reflected password from the vulnerable page. The attacker will need JavaScript access to that web page to do this.

An example of this attack is changing the field name for the password to “username” for web pages where an error is displayed when the username does not exist (e.g., “\${username} does not exist”).

For the most part, our implementation is not vulnerable to this attack. First, autofill will often fail if the attacker changes the password field’s name before autofill. Second, if the attacker changes the password field’s name after autofill, the nonce will not be replaced, as our implementation only replaces the nonce when it is in a field named the same as the field where the nonce was injected.

Still, this attack may work in cases where the attacker finds a webpage where all fields are reflected. To address this, password managers can store the URL where passwords are submitted and only replace the password when sent to that destination. This list of URLs can either be pre-generated by the password manager provider for popular websites or stored the first time the password is autofilled and transmitted by the browser (a trust-on-first-use model). This defense can be further strengthened by having the password manager learn the name of the password field, ensuring fields with different names are not autofilled. As several password managers are already checking these items before autofilling passwords [36], we believe implementing these checks to be quite practical and have included them in our list of password manager safety checks described with our design.

5.2 Deployability

To evaluate the functionality of our browser-based nonce injection implementation, we conducted tests on real websites to ensure that our implementation does not break their authentication flows. Using the Alexa top 1000 sites from May 1, 2022, we ran a Selenium script that started at the root page of each domain and traversed all links on the page up to a maximum depth of three,⁵ searching for login pages. In total, we identified login pages on 623 sites. After filtering for the subdomains of the same website, like Google and Amazon, for different countries, we were left with 573 unique login pages.

For all 573 login pages, we used a Selenium script to submit credentials from an unmodified Firefox browser and our modified version of Firefox. As illustrated in Figure 3, we set up a proxy server to record all the outgoing web requests from the browsers and save the request body. We then compared the credentials in the authentication requests sent from both browsers.

For 97% of websites (554/573), the webRequests from both browsers was identical, indicating our defense would work without any modification to the website. Of the remaining websites, 2% (11/573) calculate a hash of the password in the outgoing webRequest. Because the hash value is calculated by

⁵We selected a depth of three pages empirically by manually analyzing the top 15 domains.

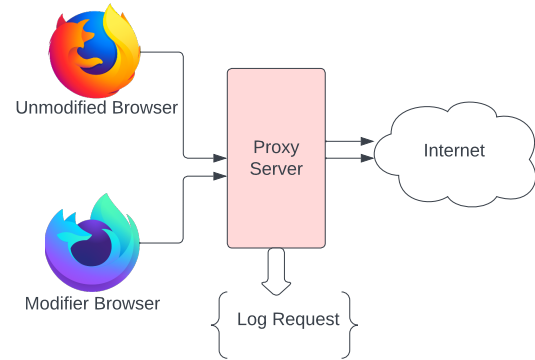


Figure 3: Functional Evaluation Architecture

JavaScript running in the DOM, when using our defense, the calculated hash will be based on the nonce. As such, any server-side integrity checks will fail and prevent the login attempt. Similarly, 1% of websites (8/573) modify the password (e.g., base64 encoded, hashed) before submitting it. This prevents our defense from detecting the nonce in the outgoing webRequest, meaning that the password will not be sent over the wire. On the websites where our defense is incompatible, the login will fail, and users will be required to manually enter the password into the login page, bypassing our defense. Overall, our measured incompatibility rate is similar to the rates identified by Stock and Johns’ [49] and by Li and Evans [29].

While the incompatibility rate of our defense is similar to the failure rate of password autofill in general [23], this rate can still be a usability annoyance [23, 37]. This annoyance could drive some users away from password autofill and could make password managers hesitant to implement our defense. In the short-term, we recommend that password managers implementing our defense maintain a list of non-compatible websites for which they will not inject password nonces, obviating this problem. As our testing shows, identifying non-compliant websites can be fully automated, making creating such a list highly feasible. In the long-term, future work can bridge the gap towards 100% compatibility (see §8.3.1).

5.3 Overhead

To measure the overhead incurred by our implementation, we examined the logs for `HttpBaseChannel` generated during our functional evaluation of our modified browser. These logs give exact measurements for each stage of the `webRequest` lifecycle, allowing us to pinpoint the processing time used by our code.

When a replacement is needed, we measured the average `webRequest` lifetime to be 4.5222 seconds. Our code accounted for 0.443 seconds, representing 10.6% of the total request duration. While this is already a minimal overhead, we also note that authentication is an uncommon event, meaning that users are unlikely to experience this overhead more than a few times a day.

This overhead can entirely be explained by the time needed to destroy and recreate the web request's body when replacing the nonce with the passwords. Based on our experience with the browser's internals and the webRequest API, there is no way to avoid this overhead without entirely re-engineering the API to change when it serializes request bodies. While this is certainly possible, our implementation tried to avoid making large-scale changes to make it more palatable for browser developers to adopt.

We also measured the overhead of our API changes when a replacement was unnecessary, finding no impact on performance (i.e., it was indistinguishable from noise).

6 Implementing in Other Browsers

The three major browser engines, Chromium (Chrome, Edge, Opera, etc.), WebKit (Safari), and Gecko (Firefox), account for 96⁶–99%⁷ of browser market share. While the codebases for each engine are distinct, porting our defense from Gecko to Chromium and WebKit is simplified by the fact that all three browser engines implement the standardized webRequest API, which forms the backbone of our defense.

6.1 Porting using the webRequest API

To port our defense to other browsers, it is necessary to implement the following functionality:

- (1) **Gather nonce replacements:** Add a `onRequestCredential` event to the webRequest API, allowing the browsers to gather nonce replacements from extensions.
- (2) **Execute nonce replacements:** Modifying the networking code to fire the `onRequestCredentials` event and modify the webRequest based on received nonce replacements.

6.1.1 Chromium.

Gather nonce replacements. Communication with the extensions will occur in a new `WebRequestEventRouter::OnRequestCredentials` method. The existing `OnBeforeRequest` method can be used as an example of how to implement this method. Within `OnRequestCredentials`, extensions with registered listeners will be called in a blocking fashion, each returning their list of requested nonce replacements (if any). After validating these nonce replacements (see §3.1), each will be stored in `WebRequestEventRouter::response_deltas` as an `EventResponseDelta` with this class being updated to support storing nonce substitutions.

To allow extensions to register a listener for `OnRequestCredentials` to call, this event will be added to the `web_request.json` file.

Execute nonce replacements. The `WebTransportHandshakeProxy::OnBeforeRequestCompleted` will be modified to replace its call to `OnBeforeSendHeaders` with a call to `OnRequestCredentials`. The completion handler for `OnRequestCredentials` will be a new `OnRequestCredentialsCompleted` method. This new method will be responsible for calling the previously replaced

`OnBeforeSendHeaders` method (and its respective `OnBeforeSendHeadersCompleted` method).

At the bottom of the `OnRequestCredentialsCompleted` function, the `ExecuteDeltas` function will handle the modification of the body. `ExecuteDeltas` will need to be modified to support nonce replacements.

6.1.2 WebKit.

Gather nonce replacements. Communications with the extensions will happen in the existing `NetworkResourceLoader::startNetworkLoad` method, after the `ResourceLoadDidSendRequest` is sent to extensions. Communication involves an inter-process communication (IPC) message being sent to the web page's process. This can be implemented by adding a message receiver in `NetworkProcessProxy.messages.in` and defining the code for that receiver in the `NetworkProcessProxy` class. This message will then be forwarded to `WebPageProxy`, followed by `WebExtensionController`, then `WebExtensionContext`. A second IPC message will be sent to the web extension's process, which must register to receive it in its `WebExtensionAPIWebRequest.messages.in` file and register a handler in `WebExtensionAPIWebRequest`. Finally, this method can call registered listeners, each of which returns its list of requested nonce replacements (if any). Valid nonce replacements (see §3.1) will be returned through this IPC call chain. This chain of passed messages can be modeled off a similar messaging chain started for `resourceLoadDidSendRequest` in `NetworkResourceLoader::startNetworkLoad`, except that the `send()` method needs to be replaced with the `sendSync()` method, allowing responses to be passed back using IPC.

To allow extensions to register a listener for the `onRequestCredentials` event, it will need to be added to `WebExtensionAPIWebRequest.idl`, with code to create the event added in `WebExtensionAPIWebRequest`.

Execute nonce replacements. Immediately after getting the list of nonce replacements, the web request's body can be directly modified within `startNetworkLoad`. This is straightforward and can follow the approach taken in our PoC Firefox implementation.

6.2 Porting For Manifest V3

Our password defense is implemented based on Manifest v2, which is supported by all three major browser engines. However, it is possible to modify the implementation of our defense to support Manifest V3. The major difference for Manifest v3 is that it is no longer possible to query extensions for responses during the webRequest workflow. This necessitates finding another mechanism for providing information on requested nonces to the code modifying the web request (which is otherwise identical to the Manifest v2 implementation).

To do this, a new API is added to the window (e.g., `window.secrets`), with a `registerNonce` method that takes as input a nonce, a secret value (i.e., the password), and a policy regarding when the nonce should be replaced by the password in outgoing requests (see §3.1). This method will store these values within the Page object. To implement this method, (i) a new `idl` file will be created to define the API, (ii) the API will be annotated to

⁶<https://gs.statcounter.com/browser-market-share>

⁷<https://www.statista.com/statistics/545520/>

require the secrets permission (or other appropriately named permission), and (iii) the registerNonce method will be annotated to receive the script state, giving it access to the Page object, (iv) a header and class file for the API will be written implementing the registerNonce functionality. Adding the IDL's is straightforward, as they are all based on the Web IDL standard. Implementing the new functionality is also straightforward, as existing IDLs provide a clear guide on how to interact with the Page object and can be used as a model for adding this functionality.

Next, code generating web requests (e.g., `FormSubmission::populateFrameLoadRequest`, `XMLHttpRequest::createRequest`, `FetchRequest::create`) will be modified to retrieve the nonce substitutions from the Page object and include them with the web request details. When the code executes, the nonce replacement is run, which retrieves the nonce substitutions from the web request information. It will be responsible for validating the replacement using its internal logic (see §3.1) and the policy given when the nonce replacement was registered.

In comparison to our initial defense, the drawback of the Manifest V3 approach (and why we choose not to implement our defense in this fashion) is that it prevents password managers from performing policy checks on the nonce replacement; thus, the browser becomes a single point of failure for policy checks. While this is not necessarily a problem, having two entities checking the nonce replacement can improve security [35].

7 A Secure Browser Channel for FIDO2

As we built, designed, and implemented our API, we hypothesized that similar APIs could be used to harden other browser-server services. Of particular interest to us was the FIDO2 protocol. Recent work has shown that this protocol is vulnerable to DOM-based attacks and malicious extensions [19, 22, 32, 56], the same adversaries our API was able to thwart in terms of password exfiltration.

As such, the third contribution of our paper is the design and implementation of a FIDO2 API, inspired by our nonce-based password replacement API, that protects FIDO2 against DOM- and malicious extension-based attacks. This demonstrates that our approach generalizes to secure other browser-server interactions. Moreover, it highlights the research importance of our first contribution: by providing a working implementation of nonce-based replacement for the browser, we have enabled research to build upon Stock and Johns' insightful protocol, where it has been otherwise stopped for the last 11 years.

7.1 FIDO2 Background

FIDO2 is a web authentication protocol that leverages public-key cryptography. A website stores the user's public key, and the corresponding private key remains with the user to sign challenges from the website during authentication. There are three participants in FIDO2 authentication:

- An **authenticator** is a hardware device that holds the user's private key. FIDO2 supports a variety of client-side authenticators, including external (remote) authenticators like hardware security keys (HSKs) and built-in platform authenticators such as biometrics. Users provide input to an

authenticator, such as a PIN or button press, to authorize it to login to a website.

- A **website** is a web application like `github.com` that offers FIDO2 as a method for 2FA or passwordless authentication. The website communicates with the authenticator through a `WebAuthn` client.
- A **WebAuthn client**, typically a web browser, acts as a relay between the website and the authenticator. To prevent phishing, the client provides the authenticator with the website's URL. The authenticator uses the credentials bound to the website to generate a response. The `WebAuthn` API in the client provides an interface to the website.

Users register their HSK as a method for 2FA or passwordless authentication. When a user clicks the Register button, the client sends a registration request to the website. Registration proceeds as follows: (1) The website first sends a registration request to the `webAuthn` client. This request provides the `webAuthn` client with a challenge, user information, and website information. (2) The `webAuthn` client sends this information to the HSK, including the website's URL and request type. (3) The HSK asks for consent from the user, typically a key tap, and then generates a new asymmetric key pair. (4) The HSK sends the credential id, public key, the website ID hash — a SHA-256 hash of RP domain or sub-domain, counter — to keep track of the count of authentication, and attestation signature: signed data object containing statements about the public key credential and the HSK make and model to the `webAuthn` client as an attestation object. (5) The `webAuthn` client then forwards this data to the website. Once the website receives the data, it verifies the signatures and critical parts of the response and, if successful, registers the HSK to the user's account.

Authentication mirrors registration, except (1) authentication omits the user information, and (2) attestation is replaced by the HSK signing the response with the private key generated during registration.

7.2 Threat Model

The threat model used in this section is based on the threat model for local attacks against FIDO2 described by Yadav and Seamons [56]. Attackers in this threat model have one of two goals: a *short-term* goal where they trick the user into validating a single login request of the attacker's choosing and a *long-term* goal where the attacker registers their authenticator in place of the user's authenticator for some account. Both attacks can be achieved if the attacker can access and modify FIDO2 values sent from the server to the authenticator or from the web page to the server. We identify two locations where a local attack can attack the FIDO2 protocol.

- (1) **Modify values passed through the DOM.** An attacker can compromise the FIDO2 protocol by injecting malicious JavaScript into the webpage, executing the protocol, and redirecting the FIDO2 request/response to an attacker-controlled authenticator. One way to achieve this is by overriding the built-in `credentials.create` and `credential.get` calls from the `webAuthn` API. The attacker can set up a virtual authenticator to register their authenticator instead of the user's [19].

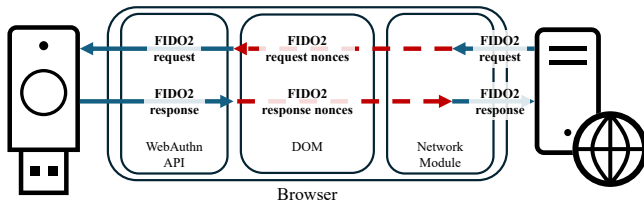


Figure 4: A diagram of our FIDO2 secure browser channel

- (2) **Modify incoming and outgoing web requests.** An attacker can also attempt to compromise the FIDO2 protocol by intercepting HTTP requests and responses. Using the data in these requests and responses, the attacker can perform the FIDO2 flow for their authenticator and then send the calculated values directly to the web server from the client’s machine.

These attacks map onto the capabilities of the *Malicious DOM Exfiltrator* and *Web Request Exfiltrator* identified in our threat model for password entry (see §2.3).

7.3 Design

To address the above attack vectors and adversaries, we present the design of a secure browser channel for the FIDO2 protocol. In our design, FIDO2 values shared through this channel are inaccessible to DOM- or extension-based scripts. At a high level, this channel is created by replacing any sensitive values in an HTTP request with nonces as soon as the browser receives them, long before the DOM or extensions can access them. The nonces are then only replaced with the actual values before being passed to the browser’s internal WebAuthn API, which is neither accessible nor modifiable by the DOM or extensions. A similar replacement happens for values returned from the authenticator. This process is diagrammed by Figure 4. More specifically, this process can be decomposed into the following five steps:

First, when sending a dummy FIDO2 registration or authenticator HTTP response, the webpage (RP) will replace the FIDO2 payload with dummy values.⁸ The real FIDO2 payload will be placed inside a `webauthn_request` header. Additionally, RP sends a `URL_resp` header that specifies the URL to which the FIDO2 response should be sent. Otherwise, the FIDO2 workflow on the server remains unchanged.

Second, when receiving an HTTP response, the browser scans it for the `webauthn_request` header. If detected, the browser will store this value internally and then strip the header from the HTTP request. Critically, this stripping operation happens before browser extensions can access the web response headers.

Third, the webpage will execute the FIDO2 workflow as normal. The FIDO2 response contains dummy values precisely so that no change is needed in how the webpage implements FIDO2 using client-side JavaScript. When this workflow interacts with the FIDO2 webAuthn API, instead of using the values passed, the values originally sent using the `webauthn_request` will be used instead. Importantly, this replacement happens within the browser,

⁸These values can be randomly generated or hard-coded.

not the web page’s DOM, preventing access to the real values by scripts running within the webpage (including scripts injected by extensions).

Fourth, when the webAuthn API returns values, these values are intercepted and stored by the browser, then replaced with randomly generated dummy values before being passed off to the webpage script that originally called the webAuthn API. The browser will also maintain a mapping between the random and real values, allowing them to be replaced later. Still, as in step 3, scripts on the webpage cannot access the real values (i.e., they are not stored anywhere within the DOM or the webpage’s JavaScript execution environment).

Fifth, when an HTTP request is made to the endpoint specified in the original `URL_resp` header, the browser will replace the FIDO2 dummy values with the appropriate real FIDO2 response values. This response will be sent to the website, completing the FIDO2 registration or authenticator process.

To safeguard FIDO2 requests and responses from interference by browser extensions, care must be taken when FIDO2 values are stripped from HTTP response headers or placed in HTTP request payloads. To this end, response headers must be stripped before `onHeaderReceived` event, the first chance that browser extensions can access headers in an HTTP response. Similarly, the real FIDO2 values must be inserted into HTTP requests after the `onSendHeaders` event, the last chance that browser extensions can read the request body.

Session accountability. In addition to the above changes, we modify the FIDO2 popup displayed by the browser to inform the user which account they are authenticating. This helps provide session accountability. While not core to the defense described above, it is easy to add with little cost.

7.4 Implementation

To validate our design’s feasibility, security, and deployability, we implemented it into Mozilla Firefox 104.0a1. In the modified Firefox, the browser examines every incoming server response in the `nsHttpChannel`, the earliest point of entry for HTTP packets in Firefox. Upon detecting a header containing the `webauthn_req` field, the browser retrieves the values of `webauthn_req` and `URL_resp`, passes them to a singleton file and subsequently removes them from the header. The dummy request in the payload undergoes the standard flow, where browser extensions can interact with it. The `WebAuthnTransactionParent` retrieves the original FIDO2 request from the singleton file, creates a `WebAuthnMakeCredentialInfo` object, and transmits it to the authenticator through the `WebAuthnManager`’s `register` function. The user interacts with the security token, and the authenticator returns the FIDO2 response. The original FIDO2 response is stored in the singleton file by the `WebAuthnTransactionParent`, while a dummy FIDO2 response is provided to the DOM. When the `nsHttpChannel` detects an outgoing web request to `URL_resp`, it retrieves the original FIDO2 response and adds it to the outgoing request’s header.

To accomplish the secure storage, we utilize the singleton design pattern, which facilitates the secure storage and redirection of FIDO2 requests and responses away from browser extensions. The

```

1 // Original code
2 response.ok(json)
3 .build();
4
5 // New code
6 response.ok(json_dummy)
7 .header("webauthn_req", json)
8 .header("URL_resp", url)
9 .build();
    
```

Listing 1: Web page changes to implement our FIDO2 design

singleton pattern restricts the instantiation of a class to a single instance within a given process. In our design, the singleton class acts as a container for requests and responses, which are stored during the early stage of the flow and later injected back into the flow at the final stage. This design simplifies access to the stored variables, as the singleton file is available anytime during the flow.

Software developers have debated the use of the singleton pattern due to its potential as a global class. However, in our specific implementation, a global class is necessary to establish a direct and efficient path for securely transferring requests and responses from the beginning to the end of a flow. Moreover, the singleton file serves as a global object, eliminating the need to pass object references between different objects and reducing the risk of coding errors.

To ensure our implementation worked as intended, we tested it against a Java-based WebAuthn Demo Server [57]. Modifying this demo server to support our design only required a small tweak to the code (see Listing 1).

To support SBC-FIDO2, we modified this code by changing the response handling as shown in Listing 1.

7.5 Security Evaluation

In our implementation, the original FIDO2 values never reach the DOM, being replaced with nonces long before the DOM could access the values. Similarly, replacement happens inside browser internals in a location that provides no access to extensions. As such, our design prevents any unauthorized access or tampering by malicious browser extensions.

To empirically confirm this analysis, we leveraged the work of Yadav and Seamons [56] to craft malicious browser libraries and extensions that seek to break the FIDO2 protocol described in our threat model. When our defense is not used, these attacks are successful, allowing for one-time malicious logins and for attackers to register their authenticator in place of the users. After our defense was enabled, both attacks immediately failed. We also built several variants of these attacks, trying to see if understanding how our implementation worked would allow us to break them, but we never found a way to access the sensitive values from any JavaScript or extension API.

7.6 Deployability Evaluation

Regarding deployability, our implementation introduces modifications to the browser architecture to establish a secure channel between websites and the authenticator. Most specifically, our design necessitates the inclusion of an additional header in the FIDO2 web request and response. While making this change is a matter of only a couple lines of code (see Listing 1), this is still a

limitation to the deployability of our approach, especially when compared to our secure browser channel for password entry, which required no changes from the majority of websites. Still, as the breadth of FIDO2 deployment is somewhat limited, now is the time to make the changes we are proposing.

As a stopgap, future research can explore approaches for automatically identifying FIDO2 values in FIDO2 requests, regardless of how they are included (e.g., in the body, as part of an object). Doing so could allow our defense to work even without changes to the server. However, for long-term implementation, we recommend adopting the headers approach due to its minimal overhead and the flexibility it offers servers.

7.7 Implementing in Other Browsers

To port our FIDO2 defense to other browsers, it is necessary to make the following changes to the codebase:

- (1) Intercept the incoming FIDO2 web response, storing the nonce values and the FIDO2 values.
- (2) At the WebAuthn API, the incoming nonce values will be replaced with the FIDO2 values. Also, values returned from the WebAuthn API will be replaced by nonces.
- (3) Intercept the outgoing FIDO2 web request, replacing the nonce values with the FIDO2 values from the WebAuthn API.

Chromium. The incoming FIDO2 web response will be intercepted and modified in the `URLLoader::BlockResponseForOrb` method (`url_loader.cc`). Changes to the WebAuthn API will happen in the `AuthenticatorImpl` class (`authenticator_impl.cc`). The outgoing FIDO2 web request will be intercepted and modified in the `URLLoader::OnBeforeSendHeaders` method. Communication between the network process and the WebAuthn API process will occur using Mojo messaging.

Webkit. The incoming FIDO2 web response will be intercepted and modified in the `NetworkResourceLoader::didReceiveResponse` method (`NetworkResourceLoader.cpp`). Changes to the WebAuthn API will happen in the `AuthenticatorGetInfoResponse` class (`AuthenticatorGetInfoResponse.cpp`) as part of the CTAP request construction. The outgoing FIDO2 web request will be intercepted and modified in the `NetworkResourceLoader::continueWillSendRequest` method. Communication between the two processes will occur using the existing IPC framework.

8 Discussion

Our design and implementation of secure browser-based authentication channels represent a significant step forward for password managers and 2FA security. With the goal of merging our APIs into Mozilla Firefox, we frequently consulted with the Mozilla Firefox developers. They provided feedback on our designs, ensuring they were suitable for use in a modern browser, and reviewed our code on several occasions. We also followed coding best practices, hoping that this would help speed up the deployment of our APIs in Firefox and other browsers. Our code base is available at <https://doi.org/10.5281/zenodo.16739763>, allowing others to replicate and extend our work.

8.1 Feedback from Product Teams

As we developed our defense and prepared this paper, we reached out to product teams to gather their thoughts on our defense. When reaching out, we provided them with an outline of our defense, a copy of this paper, and offered to answer any questions they had. As those responding to our query were not making official statements on behalf of their companies, we do not provide direct quotes nor attribute feedback to specific product teams.

Several individuals associated with the browsers respond to our queries. All were open to the idea of our defense being implemented into the browser. However, they also all mentioned that resource constraints meant it was unclear when there would be available bandwidth to implement the defense.

In their response, one browser team member mentioned a prior effort to protect credentials from DOM-based adversaries: the Digital Credentials API.⁹ They stated that this effort stumbled due to the requirement for websites to implement the defense and the lack of defense against a reflection attack. In contrast, our defense requires no changes from the majority of websites and has protections against reflection attacks (§5.1.1). Moreover, the digital credentials API does not protect against malicious extensions, which our defense does. Taken together, the existence of this API demonstrates that browser makers are interested in addressing this problem and that our defense addresses some key challenges that have stymied a prior effort.

We also had a couple of team members from a password manager respond to our queries. They indicated that one of their customers' key concerns was the ability of malicious extensions to exfiltrate passwords. The recommendation from this password manager to their customers was to address this by disallowing all extensions other than the password manager. They were very intrigued by the ability of our defense to resist attack by malicious extensions. They also recommend that we get this defense standardized through the W3C, as that provides the best pathway for mass adoption. This is something that we plan to do shortly.

8.2 Additional Applications

In this paper, we demonstrated that our proposed browser-based secure channel could be generalized to protect against local attacks against the FIDO2 protocol. Similarly, our approach of constructing a browser-based secure channel using nonce replacement can be used to ensure the integrity and confidentiality of data transmitted to and from other browser APIs, protecting against DOM- and extension-based attacks. We believe the following APIs could also be secured using our high-level design:

- The Clipboard API empowers web applications to respond to clipboard commands and seamlessly interact with the system clipboard.
- The File System Access API provides read, write, and file management capabilities, while the File API grants access to files and their contents.
- The File and Directory Entries API simulates a local file system, enabling web apps to navigate and manipulate files effortlessly.

- Geolocation API allows users to share their location with web applications, and the Media Capture and Streams API facilitates seamless audio and video media capture and streaming.

Furthermore, several experimental browser APIs can be secured from browser extensions without direct JavaScript interaction. These APIs include Web Bluetooth (for Bluetooth Low Energy device connectivity), Barcode Detection (for barcode recognition in images), WebOTP (for phone number verification using one-time passwords), Web NFC (for NFC data exchange), HID (for communication with human input/output devices), WebUSB (for exposing non-standard USB-compatible device services), MediaStream Image Capture (for capturing images and videos), and Contact Picker (for selecting and sharing limited contact details).

Other APIs could benefit from this solution. The key takeaway is that by employing the proposed defense mechanism, web servers can enhance the security of data exchange and protect against unauthorized access or tampering by browser extensions, thereby ensuring the integrity and confidentiality of sensitive information.

8.3 Future Work

Below we describe future work that could further refine the password exfiltration defense described in this paper.

8.3.1 Improving Compatibility. We have identified three approaches for improving compatibility. First, password managers could name and shame non-compliant websites. Second, password managers could implement custom handling for non-compliant websites, replicating the client-side JavaScript that breaks compatibility. Unfortunately, while both of these approaches are easy to implement, neither scale well to the long-tail of websites.

Instead, we believe that future work should explore automatically lifting relevant client-side JavaScript from login pages, then running this code during `onRequestCredentials`. This approach would involve research into (i) how to use program analysis to automatically identify relevant JavaScript (e.g., through taint analysis), (ii) modifying the browser to run this JavaScript in a locked-down environment that cannot exfiltrate the password, but still has sufficient access to a copy of the DOM to process the password, and (iii) updating the `onRequestCredentials` phase to allow for the outgoing `webRequest` to be appropriately modified.

8.3.2 Copy-And-Pasted Passwords. The password nonce designs we created can also work with passwords copied from the password manager. In this case, the password manager would copy a nonce instead of the password and then watch all browser tabs for the nonce to be pasted. At this point, the defense would work as if that nonce had been autofilled. If this feature is implemented, users will still need a way to copy raw passwords to enter them outside of the browser or use them as a fallback if nonce replacement fails. Alternatively, future research could extend nonce replacement directly to the OS, allowing for nonce replacement in apps that rely on the operating system or its libraries to send network traffic—for example, many Microsoft Store apps use the .NET Core.

8.3.3 Securing Manual Password Entry. Protecting manual password entry (i.e., users typing passwords) is challenging,

⁹<https://w3c-fedid.github.io/digital-credentials/>

particularly as an attacker can inject client-side scripts (honest-but-curious entity, DOM attacker) that listen and record keystrokes (i.e., a key logger). One possible way to secure manual password entry would be to allow the browser to enter a special password entry mode using a conditioned-safe ceremony [26]. In this mode, the browser would record the user's keystrokes, replacing each stroke with one or more random password nonce characters. The browser would also prevent any scripts or extensions from recording keystrokes while the password entry mode is active. After the user enters the password, they would leave the password entry mode.

While this approach could protect manually entered passwords, research would be needed to ensure its effectiveness. First, care must be taken when selecting the conditioned safe ceremony to ensure that users always enable it when needed. Second, it will be necessary to design the password entry mode such that it is clear when it is activated [17]. Third, research would be needed to explore how users could be made aware of and encouraged to use this functionality. Lastly, user studies would be necessary to ensure it is sufficiently usable to encourage users to adopt this functionality.

8.3.4 Denial of Service for Nonce Injection. A local attacker could attempt to sidestep our defense by injecting JavaScript that breaks the login page if it suspects a nonce has been autofilled on the page. In this case, the user may assume that there is something wrong with password autofill. In response, they may copy-and-paste or manually enter the password, unwittingly offering it up for exfiltration. To address this, we recommend implementing nonce-based replacement for copy-and-pasted (§8.3.2) and manually entered passwords (§8.3.3).

Even without these defenses, a denial of service attack isn't guaranteed to be effective. First, it requires identification of password nonces, which for users with generated passwords may be indistinguishable from the actual password. Second, injecting a denial of service attack for the login page increases the likelihood that the website, browser, or password manager can detect the local attack, allowing for it to be mitigated for all users. Such detection is even more likely in cases where generated passwords are assumed to be nonces, as this will fully block logging in for the user, triggering a report by that user of unusual behavior to the website.

8.3.5 Leveraging Nonces for Attack Detection. In our design, nonces ensure the confidentiality of credentials. However, it may also be possible to use nonces to detect attacks. For example, if the browser detects that a nonce is included in a web request body but that no appropriate substitution is made, this indicates a password exfiltration attack. The browser could then analyze a webpage's DOM for malicious scripts and potentially share this information with websites. Similarly, it could search for potentially malicious extensions exfiltrating the nonce. Alternatively, if websites can differentiate between nonces and passwords, this would also allow them to detect that an attack has occurred when they received a nonce. In either case, the fear of detection may be enough to reduce the likelihood that an attacker will attempt to steal credentials.

In either case, research will be needed to identify how the browser or website can distinguish nonces while also preventing the attacker from doing so (if the attacker realizes they are stealing a nonce, they will not steal it to avoid detection). Similarly, research will be needed to ensure that any forensic analysis happens in a way that minimizes risks to user privacy.

8.3.6 User Confusion. User confusion is a potential issue with nonce injection defenses. If users view the autofilled password, they may realize it is not their password, especially if they do not use a generated password. This could lead to confusion. Although this issue may not be that prevalent because users are less likely to investigate passwords inserted through a password manager, it is still prudent to study users' behavior when they encounter such confusion.

While the shadow DOM is not sufficiently secure to be used to show users their actual password [43], the browser could add a similar utility to securely display the real password to the user without allowing that password to be accessed by malicious client-side scripts or extensions. Future research could also investigate this potential solution.

8.4 Limitations

Our API does not protect against a compromised OS or browser, which are considered part of the trusted computing base. Similarly, we did not hide replacement values from `chrome.devtools.network`. These tools are meant to aid developers in debugging unexpected behavior, such as misbehaving nonce-replacement. This is an intentional choice, as having these APIs report the `requestBody` with a nonce instead of the password would be possible.

Our defense does not protect against attacks that occur after the password leaves the user's computer (e.g., TLS MitM). We did consider such attacks in our research, but felt that the defense needed to secure against them required too many changes from browsers, websites, and users.¹⁰

In these cases, it is important not to let perfection become the enemy of meaningful improvement. While there are still attacks that can steal passwords, the defense described provides an immediate benefit to the vast majority of password manager users on the vast majority of websites.

9 Related Work

Password managers serve to help users (a) create random, unique passwords, (b) store the user's passwords, and (c) fill in those passwords. On desktops, password managers are implemented as browser extensions. The browser does not provide any password management APIs for these extensions to use [36]. On mobile devices, there is first-party support for password managers, though this support has significant security issues [35].

9.1 Security

Password managers have the potential to provide strong security benefits but also have the potential to act as a single point of

¹⁰For more details on these threats and defenses, see the extended version of our paper at <https://doi.org/10.48550/arXiv.2509.02289>

failure for users' accounts [30, 35, 36, 47, 49]. In particular, when passwords are autofilled into websites, they are vulnerable to theft by JavaScript and extensions. In the most alarming case, if the password manager fails to require user interaction before autofilling passwords and allows passwords to be filled into iframes, it opens users to password harvesting attacks that can surreptitiously steal many if not all their passwords [36, 49]. This problem can be even worse when the operating system enforces incorrect behavior, such as in mobile devices [35].

9.2 Usability

Simmons et al. [48] systematized password manager use cases. They found that today's managers poorly support many password manager use cases. Even when supported, the tools are often targeted at experts rather than the lay users they claim to support.

Lyastani et al. [31] instrumented a password manager to collect telemetry data regarding password manager usage. Their results show that users underutilized password generation. This phenomenon was partly explained in research by Oesch et al. [37] that surveyed users and showed that many users avoid the security-critical functionality of password managers, such as password generation or password audits because they feel these features were too difficult to use. Instead, they focus on features with the highest usability, such as autofill.

9.3 Relation to Our Work

Research into the security of password managers over the last decade has consistently shown that the autofill process is an all too common element of security vulnerabilities in password managers [30, 35, 36, 47, 49]. In this paper, we explore how the autofill process can be transformed from a weakness of password managers to one of their strengths, providing benefits not available for manual entry. Critically, this security benefit is available without changing user behavior, which is not true for other password manager security benefits [37, 48].

10 Conclusion

In this paper, we explore how the password manager can actively protect passwords from theft. We take a non-implementable protocol described a decade ago by Stock and Johns' [49] and show how modern browsers can be modified to enable this protocol. Moreover, we identify a gap in this protocol's threat model and describe how it can be addressed.

To demonstrate the feasibility of these designs, we implement them in Firefox and Bitward. Using this prototype, we conduct security, functionality, and performance evaluations. We demonstrate that our tool effectively stops DOM- and extension-based password exfiltration. We also show that it does so on 97% of the Alexa top 1000 websites. This indicates that our approach will provide an immediate and significant security benefit to the vast majority of websites users employ.

Next, we show a secondary advantage of providing a working implementation of nonce-based password replacement working—it enables research on this topic to progress, enabling meaningful efforts towards securing browser-server interactions. As an example of this possibility, we extend the FIDO2 API to enable it to defend

against recently discovered local attacks [56]. This helps address a security concern in a widely used protocol and demonstrates the utility and flexibility of our high-level design for a secure browser channel using nonce replacement. We implement our design into Firefox and evaluate it, demonstrating that it empirically blocks local attacks against the FIDO2 protocol.

The code for both of our implementations can be found at <https://doi.org/10.5281/zenodo.16739763>. As such, our work advances our scientific understanding of secure password entry and makes an important practical contribution. We hope other researchers will build on this artifact to create compelling security features for password managers and password-based authentication. Future research could expand our approach to secure other authentication methods, use nonces to detect and triage vulnerabilities and secure manual password entry. Similarly, future research can explore securing other browser APIs using our design.

Acknowledgments

This material is based upon work supported by the National Science Foundation under Grants No. CNS-2226404 and CNS-1816929. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

- [1] George A. Akerlof and Robert J. Shiller. 2015. Phishing for phools. In *Phishing for Phools*. Princeton University Press.
- [2] Awake Security. 2022. Discovery of a Massive, Criminal Surveillance Campaign. <https://awakesecurity.com/blog/the-internets-new-arms-dealers-malicious-domain-registrars/>.
- [3] Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano. 2012. The quest to replace passwords: a framework for comparative evaluation of web authentication schemes. In *Proceedings of the 33rd IEEE Symposium on Security and Privacy*. IEEE, doi:10.1109/sp.2012.44.
- [4] Cristian Bravo-Lillo, Lorrie Cranor, Julie Downs, Saranga Komanduri, Stuart Schechter, and Manya Sleeper. 2012. Operating system framed in case of mistaken identity: measuring the success of web-based spoofing attacks on os password-entry dialogs. In *Proceedings of the 2012 ACM conference on Computer and communications security*. 365–377.
- [5] Cato Networks. 2022. Threat Intelligence Feeds and Endpoint Protection Systems Fail to Detect 24 Malicious Chrome Extensions. <https://www.catonetworks.com/blog/threat-intelligence-feeds-and-endpoint-protection-systems-fail-to-detect-24-malicious-chrome-extensions/>.
- [6] Google Chrome. [n. d.]. Chrome.webRequest. <https://developer.chrome.com/docs/extensions/reference/webRequest/#event-onBeforeRequest>
- [7] Chromium. 2011. 91191 - chromium - WebRequest: Access to POST data in onBeforeRequest. <https://bugs.chromium.org/p/chromium/issues/detail?id=91191> Accessed: 2023-05-03.
- [8] Chromium. 2023. The activeTab permission. <https://developer.chrome.com/docs/extensions/mv3/manifest/activeTab/> Accessed: 2023-05-03.
- [9] Chromium. 2023. chrome.declarativeNetRequest. <https://developer.chrome.com/docs/extensions/reference/declarativeNetRequest/> Accessed: 2023-05-03.
- [10] Chromium. 2023. chrome.scripting. <https://developer.chrome.com/docs/extensions/reference/scripting/> Accessed: 2023-05-03.
- [11] Chromium. 2023. chrome.webRequest. <https://developer.chrome.com/docs/extensions/reference/webRequest/> Accessed: 2023-05-03.
- [12] Chromium. 2023. Content scripts. https://developer.chrome.com/docs/extensions/mv3/content_scripts/ Accessed: 2023-05-03.
- [13] Chromium. 2023. Manifest file format. <https://developer.chrome.com/docs/extensions/mv3/manifest/> Accessed: 2023-05-03.
- [14] CVEdetails. 2024. Security vulnerabilities, CVEs, XSS, Cross Site Scripting published in January 2024. <https://www.cvedetails.com/vulnerability-list/year-2024/month-1/January.html?page=1&opxss=1&order=3&trc=318&sha=6015799bd40414f5188fe66040f69ab3dd054487>
- [15] Savino Dambra, Iskander Sanchez-Rola, Leyla Bilge, and Davide Balzarotti. 2022. When Sally Met Trackers: Web Tracking from the Users' Perspective. In *31st USENIX Security Symposium (USENIX Security 22)*. 2189–2206.

- [16] Sanchari Das, Andrew Kim, Ben Jelen, Lesa Huber, and L. Jean Camp. 2020. Non-inclusive online security: older adults' experience with two-factor authentication. In *Proceedings of the 54th Hawaii International Conference on System Sciences*.
- [17] Rachna Dhamija and J. Doug Tygar. 2005. The battle against phishing: Dynamic security skins. In *Proceedings of the 2005 symposium on Usable privacy and security*. 77–88.
- [18] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Towards measuring supply chain attacks on package managers for interpreted languages. (2020). arXiv:2002.01139 [cs.CR]
- [19] Jingjing Guan, Hui Li, Haisong Ye, and Ziming Zhao. 2022. A Formal Analysis of the FIDO2 Protocols. In *Computer Security—ESORICS 2022: 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26–30, 2022, Proceedings, Part III*. Springer, 3–21.
- [20] Feng Hao and Paul C. van Oorschot. 2022. SoK: Password-Authenticated Key Exchange—Theory, Practice, Standardization and Real-World Lessons. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. 697–711.
- [21] Cormac Herley. 2009. So long, and no thanks for the externalities: the rational rejection of security advice by users. In *Proceedings of the 2009 workshop on New security paradigms workshop*. 133–144.
- [22] Kexin Hu and Zhenfeng Zhang. 2016. Security analysis of an attractive online authentication standard: FIDO UAF protocol. *China Communications* 13, 12 (2016), 189–198.
- [23] Nicolas Huaman, Sabrina Amft, Marten Oltrogge, Yasemin Acar, and Sascha Fahl. 2021. They would do better if they worked together: The case of interaction problems between password managers and websites. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy*. IEEE. doi:10.1109/sp40001.2021.00094
- [24] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. 2018. OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks. In *Advances in Cryptology—EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part III* 37. Springer, 456–486.
- [25] Alexandros Kapravelos, Chris Grier, Neha Chachra, Christopher Kruegel, Giovanni Vigna, and Vern Paxson. 2014. Hulk: Eliciting malicious behavior in browser extensions. In *23rd USENIX Security Symposium (USENIX Security 14)*. 641–654.
- [26] Chris Karlof, J. Doug Tygar, and David A. Wagner. 2009. Conditioned-safe Ceremonies and a User Study of an Application to Web Authentication.. In *NDSS*.
- [27] Maina Korir, Simon Parkin, and Paul Dunphy. 2022. An empirical study of a decentralized identity wallet: Usability, security, and perspectives on user control. In *Proceedings of the 18th Symposium on Usable Privacy and Security*. USENIX, Boston, MA. <https://www.usenix.org/conference/soups2022/presentation/mayer>
- [28] Robert Lemos. 2021. Dependency problems increase for open source components. <https://www.darkreading.com/application-security/dependency-problems-increase-for-open-source-components/d/d-id/1340665>
- [29] Hannah Li and David Evans. 2017. Horcrux: A password manager for paranoids. (2017). arXiv:1706.05085 [cs.CR]
- [30] Zhiwei Li, Warren He, Devdatta Akhawe, and Dawn Song. 2014. The Emperor's New Password Manager: Security Analysis of Web-based Password Managers. In *USENIX Security Symposium*. 465–479.
- [31] Sanam Ghorbani Lyastani, Michael Schilling, Sascha Fahl, Michael Backes, and Sven Bugiel. 2018. Better managed than memorized? Studying the impact of managers on password strength and reuse. In *Proceedings of the 28th USENIX Security Symposium*. USENIX.
- [32] Ahmed Tanvir Mahdad, Mohammed Jubur, and Nitesh Saxena. 2024. Breaching Security Keys without Root: FIDO2 Deception Attacks via Overlays Exploiting Limited Display Authenticators. In *31st ACM Conference on Computer and Communications Security (CCS)*.
- [33] Mozilla. 2017. 1376155 - webrequest: Support modifying request bodies (e.g. via requestbody blockingresponse). https://bugzilla.mozilla.org/show_bug.cgi?id=1376155 Accessed: 2023-05-03.
- [34] Mark Munder. 2017. PSA: 4.8 Million Affected by Chrome Extension Attacks Targeting Site Owners. <https://www.wordfence.com/blog/2017/08/chrome-browser-extension-attacks/>
- [35] Sean Oesch, Anuj Gautam, and Scott Ruoti. 2021. The emperor's new autofill framework: a security analysis of autofill on iOS and Android. In *Proceedings of the 37th Annual Computer Security Applications Conference*. ACM. doi:10.1145/3485832.3485884
- [36] Sean Oesch and Scott Ruoti. 2020. That was then, this is now: a security evaluation of password generation, storage, and autofill in browser-based password managers. In *Proceedings of the 30th USENIX Security Symposium*. USENIX.
- [37] Sean Oesch, Scott Ruoti, James Simmons, and Anuj Gautam. 2022. "It basically started using me:" An observational study of password manager usage. In *Proceedings of the 40th ACM CHI Conference on Human Factors in Computing Systems*. ACM. doi:10.1145/3491102.3517534
- [38] Marc Ohm, Henrik Plate, Arnold Sykosh, and Michael Meier. 2020. Backstabber's knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings* 17. Springer, 23–43.
- [39] Mark O'Neill, Scott Ruoti, Kent Seamons, and Daniel Zappala. 2016. TLS proxies: friend or foe?. In *Proceedings of the 17th ACM Internet Measurement Conference*. ACM. doi:10.1145/2987443.2987488
- [40] OSITCOM. 2021. Google Removes 500 plus Malicious Chrome Extensions. <https://www.ositcom.com/61>.
- [41] OWASP. 2022. Cross site scripting (XSS). <https://owasp.org/www-community/attacks/xss/> Accessed: 2023-05-03.
- [42] Nikolaos Pantelaios, Nick Nikiforakis, and Alexandros Kapravelos. 2020. You've changed: Detecting malicious browser extensions through their update deltas. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 477–491.
- [43] Scott Ruoti, Jeff Andersen, Tyler Monson, Daniel Zappala, and Kent Seamons. 2016. MessageGuard: A Browser-based Platform for Usable, Content-Based Encryption Research. arXiv:1510.08943 [cs.CR]
- [44] Scott Ruoti and Kent Seamons. 2017. End-to-end passwords. In *Proceedings of the 20th New Security Paradigms Workshop*. ACM. doi:10.1145/3171533.3171542
- [45] Help Net Security. 2021. Why XSS is still an XXL issue in 2021. <https://www.helpnetsecurity.com/2021/06/15/xss-attacks/>
- [46] Asuman Senol, Gunes Acar, Mathias Humbert, and Frederik Zuiderveen Borgesius. 2022. Leaky Forms: A Study of Email and Password Exfiltration Before Form Submission. In *31st USENIX Security Symposium (USENIX Security 22)*. 1813–1830.
- [47] David Silver, Suman Jana, Dan Boneh, Eric Yawei Chen, and Collin Jackson. 2014. Password Managers: Attacks and Defenses.. In *USENIX Security Symposium*. 449–464.
- [48] James Simmons, Oumar Diallo, Sean Oesch, and Scott Ruoti. 2021. Systematization of password manager use cases and design paradigms. In *Proceedings of the 37th Annual Computer Security Applications Conference*. ACM. doi:10.1145/3485832.3485889
- [49] Ben Stock and Martin Johns. 2014. Protecting users against XSS-based password manager abuse. In *Proceedings of the 9th ACM symposium on Information, computer and communications security*. ACM, 183–194.
- [50] Sansec Forensics Team. 2024. Polyfill supply chain attack hits 100K+ sites. <https://sansec.io/research/polyfill-supply-chain-attack>
- [51] Alanna Titterington. 2023. Dangerous Browser Extensions. <https://usa.kaspersky.com/blog/dangerous-browser-extensions-2023/29546/>
- [52] W3. 2023. HTML - Construct Entry List. <https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#constructing-the-form-data-set>
- [53] WHATWG. 2023. HTML standard - WHATWG. <https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#form-submission-algorithm> Accessed: 2023-05-03.
- [54] WhiteSource. 2022. Remediating vulnerabilities in NPM Packages - WhiteSource. <https://www.mend.io/resources/research-reports/mend-research-report-remediating-vulnerabilities-in-npm-packages/>
- [55] Thomas D. Wu et al. 1998. The Secure Remote Password Protocol. In *NDSS*, Vol. 98. Citeseer, 97–111.
- [56] Tarun Kumar Yadav and Kent Seamons. 2024. A Security and Usability Analysis of Local Attacks Against FIDO2. In *Network and Distributed System Security Symposium (NDSS)*.
- [57] Yubico. [n. d.]. Java WebAuthn Server. <https://github.com/Yubico/java-webauthn-server>